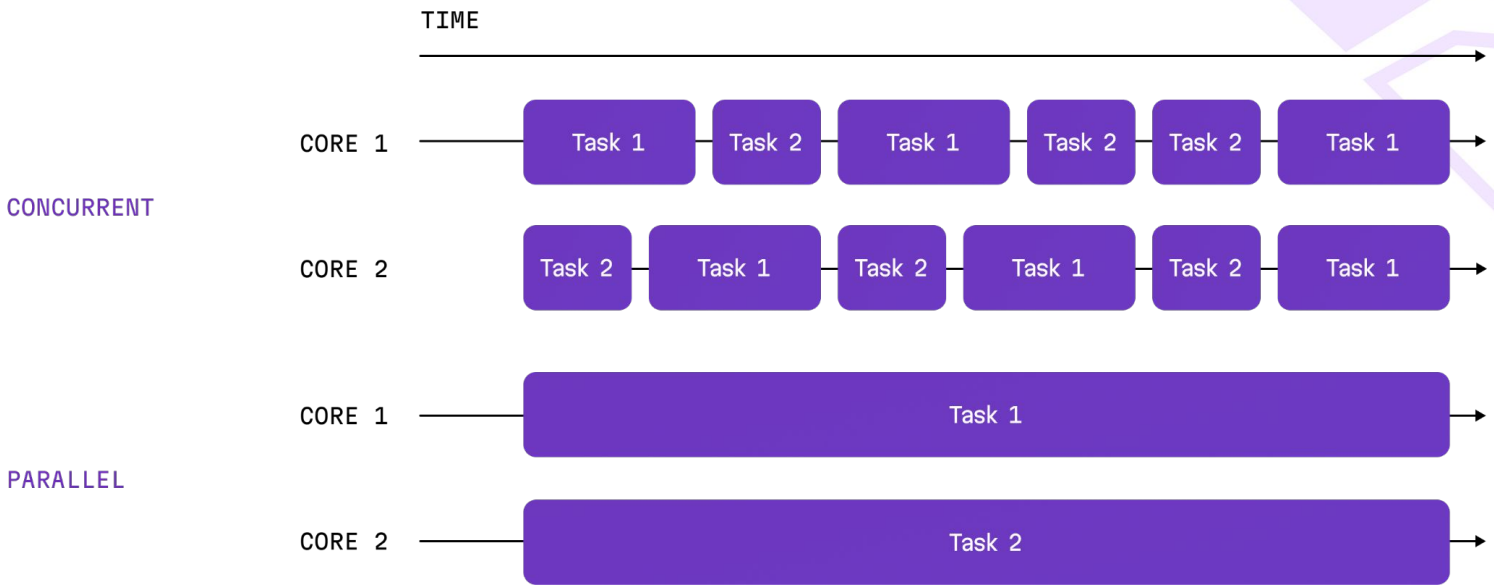
Unlocking Concurrency and Performance in Python with Async I/O

Jothir Adithyan
Software Engineer

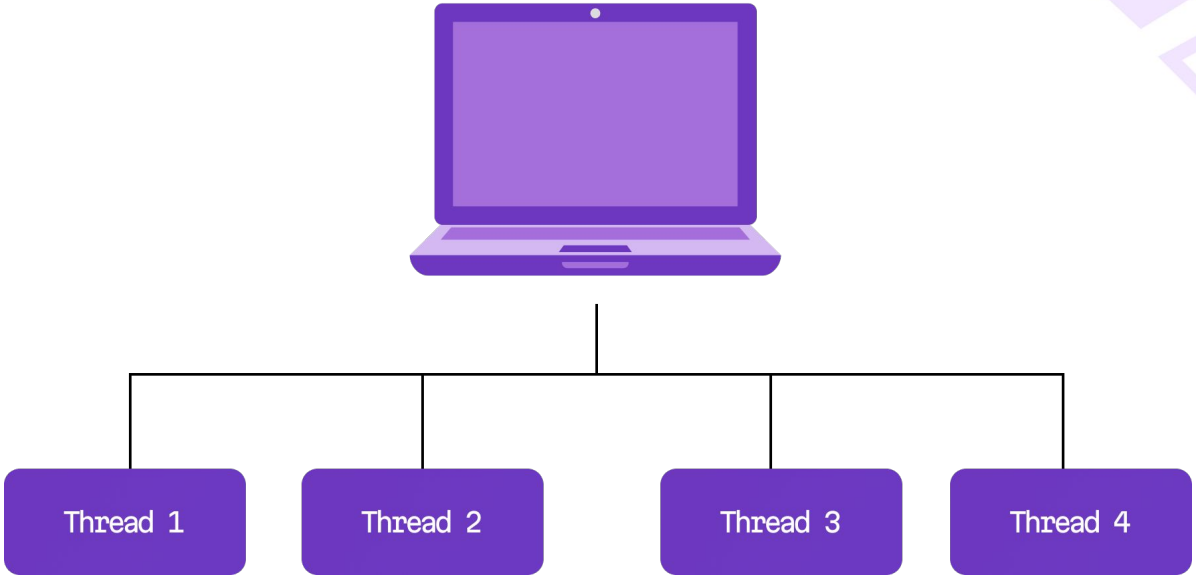
INDEX

1. Introduction to **AsyncIO**
2. **AsyncIO** breakdown
3. Performance
4. **Best practices**

Parallelism vs Concurrency

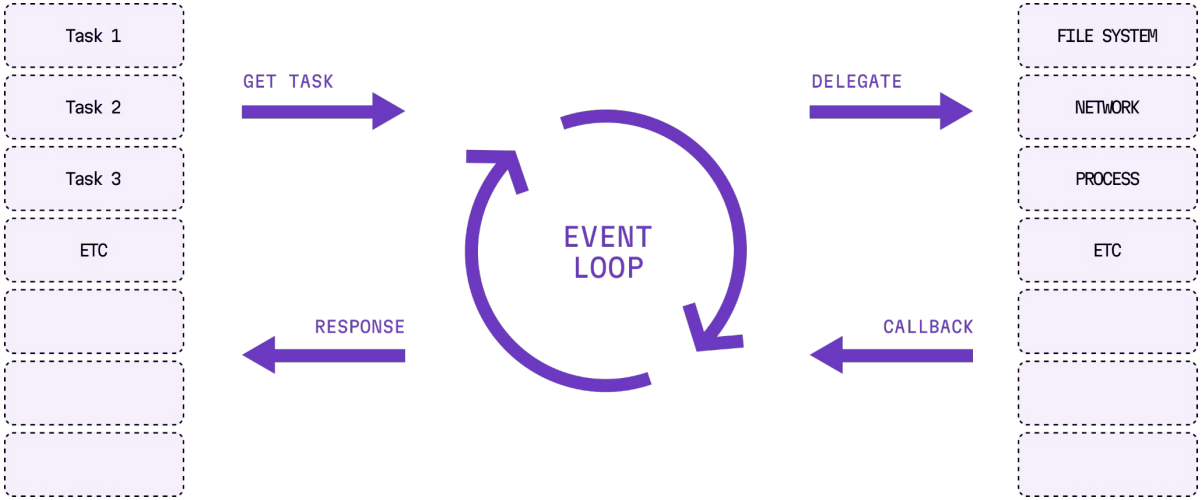


Multi Threading



Goal of AsyncIO

1. Optimize different tasks efficiently
2. Using a single thread



Deep dive into AsyncIO

Life before async await

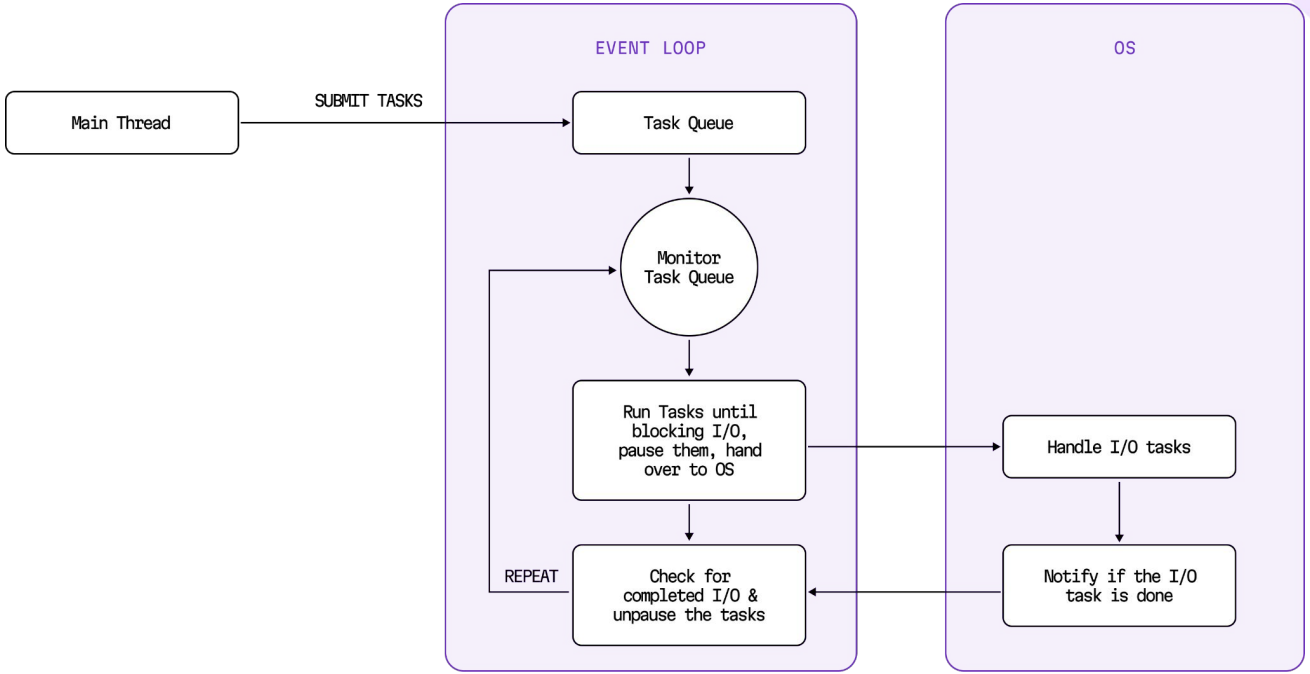
```python
import asyncio

@asyncio.coroutine
def my_coroutine():
    print("Start")
    yield from asyncio.sleep(2)
    print("End")

@asyncio.coroutine
def main():
    yield from my_coroutine()

# Event loop management
loop = asyncio.get_event_loop()
loop.run_until_complete(main())
loop.close()
```

<=Python 3.4

```python
import asyncio

async def my_coroutine():
    print("Start")
    await asyncio.sleep(2)
    print("End")

async def main():
    await my_coroutine()

asyncio.run(main())
```

>=Python 3.5

Event Loop

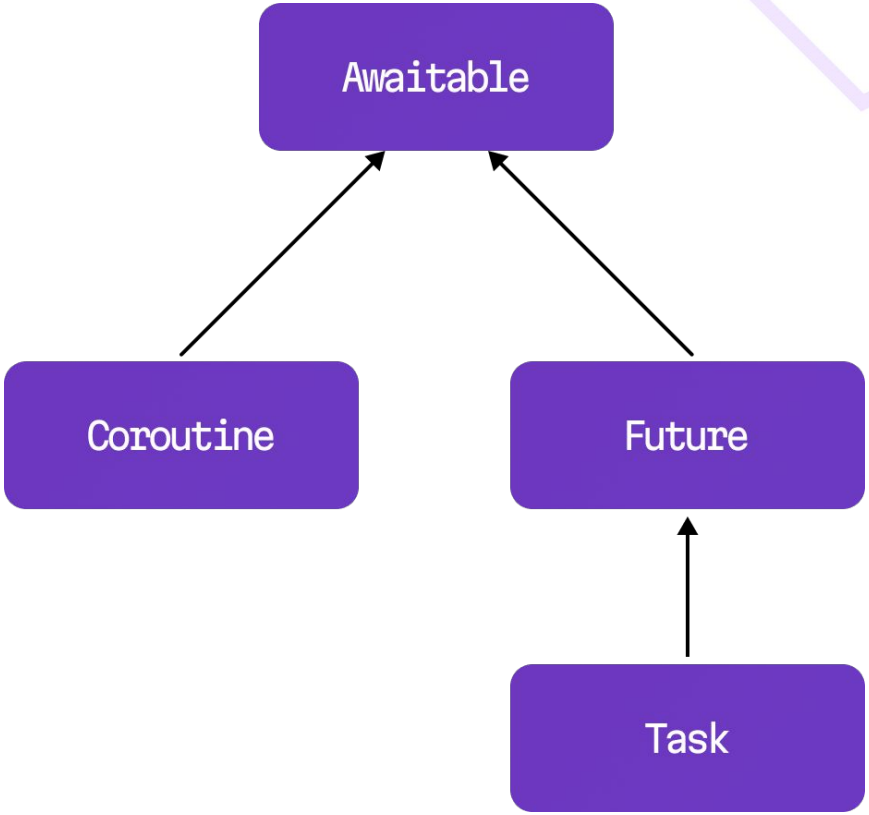

EVENT LOOP
OS
Main Thread
SUBMIT TASKS
Task Queue
Monitor Task Queue
Run Tasks until blocking I/O, pause them, hand over to OS
Handle I/O tasks
REPEAT
Check for completed I/O & unpause the tasks
Notify if the I/O task is done

Types of Loops – Selector Event Loop

- Relies on OS APIs
  - select()

    ready_r_fds, ready_w_fds, ready_exc_fds = select (r_fds, w_fds, exc_fds)
  - poll()
  - epoll()
  - kqueue()
- Monitors file descriptors and schedules tasks when they are ready.
- For UNIX-like systems (Linux, macOS).
- Works on Windows but with limitations.

AsyncIO Hierarchy


Awaitable
Coroutine
Future
Task

Awaitable

- The Awaitable class has an abstract method `__await__()`.
- `__await__()` enables the use of await keyword

```python
import asyncio
from typing import Awaitable

class SimpleAwaitable(Awaitable):
    def __init__(self, value):
        self.value = value

    def __await__(self):
        yield from asyncio.sleep(1).__await__()  # Simulate an async operation
        return self.value  # Return the awaited value

async def main():
    result = await SimpleAwaitable(42)
    print(result) # 42

asyncio.run(main())
```

Future

- A future is an object that returns a value later in the future but not now.

- A future object is the result of an asynchronous operation.

```python
import asyncio
from asyncio import Future


async def main():
    my_future = Future()
    print(my_future.done())  # False

    my_future.set_result("Bright")

    print(my_future.done())  # True

    print(my_future.result()) # Bright


asyncio.run(main())
```

```python
import asyncio
from asyncio import Future


async def main():
    my_future = Future()
    print(my_future.done())  # False

    my_future.set_exception(
        ValueError("Oops! Something went wrong!")
    )

    print(my_future.done())  # True

    try:
        print(my_future.result())  # Error Thrown
    except ValueError as e:
        print(e)  # Oops! Something went wrong!


asyncio.run(main())
```

Coroutine

- A function that can pause its execution when encountering an operation that may take a while to complete.
- While paused, other code can run, enabling asynchronous execution and better performance.

Coroutine: Async, Await

- To create and pause a coroutine, you use the Python async and await keywords:

- The async keyword creates a coroutine.
- The await keyword pauses a coroutine.

```python
async def square(number: int) -> int:
    return number * number


result = square(10)
print(result)  # <coroutine object square at 0x10501c940>
# sys:1: RuntimeWarning: coroutine 'square' was never awaited
```

```python
import asyncio

async def square(number: int) -> int:
    return number * number

async def main():
    result = await square(10)
    print(result) # 100

asyncio.run(main())
```

Task

- Subclass of Future class.
- Represents a coroutine executed by the event loop.
- A way to schedule a coroutine to run asynchronously.

```python
import asyncio
from asyncio import Task

async def my_coroutine():
    print("Coroutine started!")
    try:
        await asyncio.sleep(5)
        print("Coroutine completed!")
        return "Success"
    except asyncio.CancelledError:  # You can handle this in await task
        print("Coroutine was cancelled!")
        return "Failed"

async def main():
    task: Task = asyncio.create_task(my_coroutine())
    # Coroutine started!

    await asyncio.sleep(2)  # Let's wait a while
    print(task.done())  # False

    task.cancel()
    print(task.cancelled())  # False

    result = await task  # Coroutine was cancelled!

    print(result)  # Failed
    print(task.result())  # Failed
    print(task.done())  # True

# Run the main coroutine
asyncio.run(main())
```

asyncio.TaskGroup()

If one task fails, the TaskGroup cancels the remaining tasks and raises the exception.

```python
import asyncio

async def task(name, delay):
    print(f"{name} started")
    await asyncio.sleep(delay)
    if name == "Task 3":
        raise ValueError(f"{name} encountered an error!")
    print(f"{name} completed")
    return f"{name} result"

async def taskgroup_example():
    try:
        async with asyncio.TaskGroup() as tg:
            t1 = tg.create_task(task("Task 1", 2))
            t2 = tg.create_task(task("Task 2", 5))
            t3 = tg.create_task(task("Task 3", 4))
            t4 = tg.create_task(task("Task 4", 1))
    except ExceptionGroup as e:
        print(f"An error occurred: {e} {e.exceptions}")

    for current_task in [t1, t2, t3, t4]:
        try:
            print(current_task.result())
        except BaseException as e:
            print(f"An error occurred: {type(e)} {e}")

asyncio.run(taskgroup_example())

>> Task 1 started
>> Task 2 started
>> Task 3 started
>> Task 4 started
>> Task 4 completed
>> Task 1 completed
>> An error occurred: unhandled errors in a TaskGroup (1 sub-exception)
(ValueError('Task 3 encountered an error!'),)
>> Task 1 result
>> An error occurred: <class 'asyncio.exceptions.CancelledError'>
>> An error occurred: <class 'ValueError'> Task 3 encountered an error!
>> Task 4 result
```

asyncio.gather(...)

Similar to TaskGroup()

But return_exceptions = True,
Waits for all to complete even if error occurs.

```python
import asyncio

async def task(name, delay):
    print(f"{name} started")
    await asyncio.sleep(delay)
    if name == "Task 3":
        raise ValueError(f"{name} encountered an error!")
    print(f"{name} completed")
    return f"{name} result"


async def gather_example():
    task_results = await asyncio.gather(
        task("Task 1", 2),
        task("Task 2", 3),
        task("Task 3", 1),
        task("Task 4", 4),
        return_exceptions=True,
    )
    print(task_results)

asyncio.run(gather_example())

>> Task 1 started
>> Task 2 started
>> Task 3 started
>> Task 4 started
>> Task 1 completed
>> Task 2 completed
>> Task 4 completed
>> ['Task 1 result', 'Task 2 result', ValueError('Task 3 encountered an
error!'), 'Task 4 result']
```

Performance

Impact of asyncio in everyday tasks

Sync Vs Async – I/O operation

```python
import time
from time import perf_counter

def sync_task(task_id, delay):
    print(f"Task {task_id} started (sync)...")
    time.sleep(delay)  # Simulates a blocking I/O operation
    print(f"Task {task_id} completed (sync)")

def sync_main():
    start_time = perf_counter()
    for i in range(5):  # Simulate 5 tasks
        sync_task(i, 2)  # Each task takes 2 seconds
    end_time = perf_counter()
    print(f"Synchronous execution took {end_time - start_time:.2f} seconds.")

sync_main()

>> Task 0 started (sync)...
>> Task 0 completed (sync)
>> Task 1 started (sync)...
>> Task 1 completed (sync)
>> Task 2 started (sync)...
>> Task 2 completed (sync)
>> Task 3 started (sync)...
>> Task 3 completed (sync)
>> Task 4 started (sync)...
>> Task 4 completed (sync)
>> Synchronous execution took 10.00 seconds.
```

```python
import asyncio
from time import perf_counter

async def async_task(task_id, delay):
    print(f"Task {task_id} started (async)...")
    await asyncio.sleep(delay)  # Simulates a non-blocking I/O operation
    print(f"Task {task_id} completed (async)")

async def async_main():
    start_time = perf_counter()
    tasks = [async_task(i, 2) for i in range(5)]  # Simulate 5 tasks
    await asyncio.gather(*tasks)  # Run all tasks concurrently
    end_time = perf_counter()
    print(f"Asynchronous execution took {end_time - start_time:.2f} seconds.")

>> Task 0 started (async)...
>> Task 1 started (async)...
>> Task 2 started (async)...
>> Task 3 started (async)...
>> Task 4 started (async)...
>> Task 0 completed (async)
>> Task 1 completed (async)
>> Task 2 completed (async)
>> Task 3 completed (async)
>> Task 4 completed (async)
>> Asynchronous execution took 2.00 seconds
```

Best Practices for Async in Python

Using aiohttp for Asynchronous HTTP Requests

Efficient HTTP Handling:

- Asynchronous HTTP client/server library for handling requests without blocking.

Use Case:

- Ideal for APIs, web scraping, and real-time communication.

```python
import aiohttp
import asyncio

async def fetch_url(session, url):
    async with session.get(url) as response:
        return await response.text()

async def main():
    async with aiohttp.ClientSession() as session:
        html = await fetch_url(session, "https://example.com")
        print(html)

asyncio.run(main())
```

Use aiofiles for Async File Handling

Why aiofiles?

- Traditional file operations block the event loop.
- aiofiles allows non-blocking file operations for improved performance.

Common Use Case:

- Reading/writing large files or logs in async applications.

```python
import aiofiles
import asyncio

async def write_file():
    async with aiofiles.open("example.txt", mode="w") as f:
        await f.write("Hello, aiofiles!\n")
        await f.write("This is an async file write
operation.")
asyncio.run(write_file())

```

“In async programming, patience is the key, but only for the task, not for you.”

— AI, ChatGPT

References

- Coroutine, Future, Tasks - Pythontutorial.net
- WSGI vs ASGI - Medium
- Best Practises - discuss.python.org

loop.close()

Thank you for listening

Jothir Adithyan
Product Engineer

AdithyanJothir

adithyanjothir